

1 - Paradigmi di Programmazione e Astrazione

Dal cosa al come

La scienza del software studia il passaggio dal "cosa" (quello che il programmatore pensa, quindi il problema) al "come" (come viene eseguito dalla macchina)

L'estremo "cosa" raccoglie gli obbiettivi, i desideri e i bisogni dell'utente, la sua conoscenza del dominio del problema, tutti espressi in linguaggio naturale

L'estremo "come" è l'hardware che può raggiungere quegli obbiettivi e soddisfare i bisogni e i desideri in modo estremamente procedurale e rigorosamente sequenziale

La scienza della programmazione ha esplorato molti punti dello spettro "what-how", creando diversi **paradigmi di programmazione**.

Paradigma

Paradigma in generale

Complesso di regole metodologiche, modelli esplicativi, criteri di soluzione di problemi che caratterizza una comunità di scienziati in una fase determinata dell'evoluzione storica della loro disciplina

Paradigma di programmazione

Collezioni di modelli concettuali che insieme plasmano il processo di analisi, progettazione e sviluppo di un programma

Questi modelli concettuali “strutturano” il pensiero in quanto determinano la forma di programmi validi, influenzano il modo in cui pensiamo e formuliamo le soluzioni, arrivando a condizionare perfino la possibilità di trovare una soluzione.

Un paradigma cambia fundamentalmente il modo in cui guardiamo ai problemi incontrati nel passato, ci deve dare un nuovo schema per pensare ai problemi futuri e cambia le nostre priorità, le nostre idee su quanto merita attenzione e su cosa sia importante, cambiando effettivamente punto di vista

Un nuovo paradigma è spesso **introdotto per risolvere un particolare problema**, ma si rivela poi adatto per altri. Per un problema degli anni 60, ci si è accorti che nessun paradigma esistente non era effettivamente utilizzabile ed è stato inventato il **paradigma ad oggetti (object oriented)**

Nel senso della macchina di Turing, tutti i **linguaggi di programmazione più comuni sono universali**, tuttavia ogni linguaggio di programmazione si basa, o meglio supporta, un particolare paradigma, fornendo:

- Le **primitive** di quel paradigma
- I **metodi di composizione** di quel paradigma
- Un linguaggio utente appropriato che rende chiari i programmi scritti secondo quel paradigma
- Un'esecuzione efficiente dei programmi scritti nel linguaggio scelto al punto precedente

Per esempio il C è un linguaggio basato sul **paradigma imperativo**

Rapporto Paradigma-Linguaggi

I linguaggi di programmazione sono dotati di opportuni **costrutti linguistici** che riflettono i modelli concettuali di un paradigma, al fine di facilitare l'espressione di una soluzione definita attraverso i modelli concettuali dello stesso paradigma. I linguaggi di programmazione possono supportare **più di un paradigma**.

Categorie di paradigmi

1. Paradigmi che supportano tecniche di programmazione di basso livello (assembly)
2. Paradigmi che supportano metodi di progettazione di algoritmi (e.g., divide-et-impera, programmazione dinamica)
3. Paradigmi che supportano gli approcci di alto livello alla programmazione (e.g., paradigma funzionale e quello basato su regole)

Approccio dei paradigmi al problem solving

I paradigmi che supportano la programmazione ad alto livello possono essere raggruppati in base al loro approccio alla soluzione dei problemi.

- **Approccio operativo:** descrive per passi **COME** costruire una soluzione.
- **Approccio dimostrazionale:** è una variante del precedente che illustra la soluzione in modo operativo per esempi specifici e lascia al sistema il compito di generalizzare queste soluzioni di esempi ad altri casi.
- **Approccio definizionale:** esso stabilisce delle proprietà della soluzione in modo da vincolarla senza descrivere come viene calcolata.

Approccio operativo

Questo approccio è il pilastro della programmazione imperativa e qui il programmatore agisce come un coordinatore che impartisce ordini sequenziali. **Il focus è sul mutamento di stato del sistema (le istruzioni modificano dei valori sulla memoria) attraverso istruzioni precise.**

Il concetto chiave di questo approccio è il programmatore controlla quel **flusso di controllo** (cicli for, istruzioni if, assegnazione di variabili).

Approccio dimostrazionale

È una variante dell'operazionale, ma più "guidata dagli esempi", come quelli di input/output. Il sistema generalizza automaticamente la soluzione, quindi tipico di contesti in cui si "insegna" al sistema cosa fare (come il Machine Learning).

Il concetto chiave è che **non scrivi l'algoritmo, mostri cosa deve fare**

Approccio definizionale

Si descrive **COSA** deve essere vero, non come ottenerlo, specificando proprietà, vincoli o relazioni. Il sistema trova autonomamente una soluzione che le soddisfi. Il concetto chiave è **che si descriva il risultato ma non il suo procedimento**

Approccio	Concetto chiave	Focus	Analogia
Operazionale	Ecco le istruzioni che devi seguire	Sequenza di passi	Seguire una ricetta passo-passo
Dimostrazionale	Guarda come si fa	Esempi specifici	Mostrare a qualcuno come si prepara una ricetta
Definizionale	Leggi cosa voglio ottenere	Proprietà e vincoli	Ordinare un piatto al ristorante, descrivere cosa vuoi non ti interessa il come

La scienza della programmazione

Chi studia questa scienza si deve occupare di diversi campi:

- **I metodi di analisi dei problemi**, che consentono di passare da formulazioni spesso imprecise ed ambigue di un problema, a specifiche espresse in un qualche linguaggio formale
- **Le tecniche di trasformazione dei programmi**, o tecniche di programmazione, che trasformano la specifica di un problema in un programma che lo risolve.
- **I metodi di verifica della correttezza**, che affrontano il problema di verificare la terminazione dei programmi e la rispondenza alle specifiche date

- **La teoria della complessità computazionale**, che studia l'efficienza delle soluzioni trovate
- **La teoria dei linguaggi formali e traduttori**, che studia gli strumenti utilizzabili per comunicare un algoritmo ad un elaboratore.

Approccio multi-paradigmatico

Nessun singolo paradigma è appropriato per tutti i problemi, le applicazioni complesse necessitano l'adozione di più paradigmi di programmazione.

Ogni singolo paradigma di programmazione è caratterizzato da:

- Un diverso **potere di elisione**, cioè capacità di esprimere qualcosa in modo conciso (compattare per esempio il codice usando un paradigma rispetto ad un altro)
- Una diversa **invarianza rispetto ai cambiamenti** apportati nella strategia di soluzione di un problema (stesso paradigma per modifiche successive)

Occorre sempre bilanciare i costi dovuti all'uniformità di paradigma con i costi determinati dall'uso di diversi paradigmi per un medesimo problema. I costi che si riscontrano sono:

- Costi iniziali di apprendimento (il programmatore deve essere formato)
- Costi di debugging
- Costi di variazione dovuti all'evoluzione del programma
- Costi di esecuzione dell'applicazione (un paradigma sbagliato potrebbe portare ad un'esecuzione più lenta per un codice più pesante)

Classificazioni di applicazioni software rispetto alla natura degli elementi

- **Applicazioni orientate alla realizzazione di funzioni**: la complessità prevalente riguarda le funzioni da realizzare (come i programmi utilizzati dagli utenti nella quotidianità).
- **Applicazioni orientate alla gestione dei dati**: l'aspetto prevalente è rappresentato dai dati che vengono memorizzati, ricercati e modificati, e che costituiscono il patrimonio informativo di una organizzazione.
- **Applicazioni orientate al controllo**: la complessità prevalente del sistema riguarda il controllo delle attività che si sincronizzano e cooperano durante l'evoluzione del sistema.

Classificazioni di applicazioni software rispetto al flusso di esecuzione delle attività

- **Applicazioni sequenziali**: sono caratterizzate da un unico flusso di controllo che governa l'evoluzione dell'applicazione. Queste sono le applicazioni più tradizionali, e vengono spesso adottate come riferimento per i metodi e le tecniche di base per la progettazione.
- **Applicazioni concorrenti**: sono caratterizzate dal fatto che le varie attività che compongono il sistema non hanno una natura inerentemente sequenziale, ma

necessitano di meccanismi di sincronizzazione e comunicazione.

- **Applicazioni dipendenti dal tempo:** sono influenzate da vincoli temporali riguardanti sia la velocità di esecuzione delle attività sia la necessità di sincronizzare le attività stesse.

Paradigmi operazionali

I paradigmi operazionali si basano su sequenze computazionali calcolate passo dopo passo. Nei paradigmi operazionali è difficile stabilire se l'insieme dei valori calcolati operativamente è proprio l'insieme dei valori soluzione. Le tecniche di verifica e di debugging cercano di superare questo problema di programmazione e spesso ci si accontenta di stabilire che i due insiemi siano "sufficientemente vicini", ovvero indistinguibili per la sottoclasse attesa di problemi effettivi

Side-effecting

I paradigmi operazionali si distinguono in:

- **Side-effecting:** procedono modificando ripetutamente la loro rappresentazione dei dati (le variabili sono legate a locazioni di memoria), come il C
- **Non-side-effecting:** procedono creando continuamente nuovi dati. Questi paradigmi includono quelli che tradizionalmente sono detti **funzionali** (si dà ad una funzione dei valori e lei produce un risultato, non modificando valori esistenti ma calcolandone altri e creandone copie eventuali). Una volta allocato in memoria, un dato non può più essere modificato. Una "funzione pura" prende in input dei valori e ne restituisce di nuovi, senza mai intaccare gli originali né alterare lo stato esterno.

I paradigmi operazionali "side-effecting" si distinguono in:

- Imperativi
- Orientati agli oggetti

Sequenziale vs Concorrente

Per ognuno dei paradigmi visti precedentemente possiamo distinguere due versioni:

- **Sequenziale:** il flusso di controllo è unico
- **Concorrente o parallelo:** più flussi di controllo sono ammessi.

Astrazione nella progettazione

Definizione di astrazione

L'astrazione, in ambito scientifico, significa cambiare la rappresentazione di un problema, "trascinando via" un concetto, una idea, un principio da una realtà concreta.

L'obiettivo del cambio di rappresentazione è quello di concentrarsi su aspetti rilevanti dimenticando gli elementi secondari.

Non si tratta di omettere parti della rappresentazione di un problema, ma di riformulare lo stesso concentrando l'attenzione su idee generali piuttosto che su manifestazioni specifiche di quelle idee, tenendo conto della prospettiva di un osservatore.

☰ Esempio di astrazione

Problema: trovare il percorso migliore per un commesso viaggiatore che deve visitare diverse città.

Se 'migliore' significa 'più breve', possiamo trascurare fattori come presenza di traffico e qualità della viabilità, e riformulare il problema come ricerca di minimo percorso in un grafo.



L'astrazione si focalizza sulle caratteristiche essenziali di un oggetto a rispetto della prospettiva di colui che osserva.

Astrazione: processo o entità

Il termine astrazione sotto-intende due concetti distinti, entrambi validi e ugualmente necessari:

- **Un processo:** l'estrazione delle informazioni essenziali e rilevanti per un particolare scopo, ignorando il resto dell'informazione
 - **Una entità:** una descrizione semplificata di un sistema che enfatizza alcuni dei dettagli o proprietà trascurandone altri
- Entrambe le viste sono valide e di fatto necessarie.

☰ Esempio dell'aereo

Si suppone che si voglia controllare il traffico aereo, avremo due tipologie di dettagli:

- Dettagli **essenziali**: posizione del velivolo, velocità, etc.
- Dettagli **irrilevanti**: colore, nomi dei passeggeri, etc.

Nel quotidiano il principio di astrazione è costantemente applicato ogni qualvolta utilizziamo uno strumento senza per questo sapere come è realizzato.

Astrazione nel software

Similmente, nella programmazione l'astrazione allude alla distinzione che si fa tra

- **Cosa** (what) fa un pezzo di codice
- **Come** (how) esso è implementato

Per l'utente l'essenziale è cosa fa il codice, mentre non è interessato ai dettagli dell'implementazione.

Astrazione funzionale

Definizione

L'astrazione funzionale si riferisce alla progettazione del software, e in particolare alla possibilità di specificare un modulo software che trasforma dei dati di input in dati di output nascondendo i dettagli algoritmici della trasformazione.

Il modulo software deve trasformare un input in un output, cioè deve calcolare una funzione, senza che sia visibili al fruitore del modulo i vari dettagli della trasformazione, lasciandogli conoscere soltanto le corrette convenzioni di chiamata (**specificata sintattica**) e cosa fa il modulo (**specificata semantica**)

Esempio: modulo che realizza un operatore per il calcolo del fattoriale

La **specificata sintattica** indica il nome del modulo (e.g., **fatt**), il tipo di dato in input (un intero) e il tipo di risultato (un intero), in modo da permettere la corretta chiamata del modulo. **fatt(intero) → intero**

La **specificata semantica** indica la trasformazione operata, cioè la funzione calcolata:

$$fatt(n) = \begin{cases} n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$$

Come la trasformazione è calcolata o **realizzata** (e.g., iterativamente o ricorsivamente) non è noto al fruitore del modulo.

Ma come si specifica la semantica del modulo?

Un modo è quello di esprimere, mediante due predicati, la relazione che lega i dati di ingresso ai dati di uscita:

- Se il primo predicato (detto **precondizione**) è vero sui dati di ingresso e se il programma termina su quei dati;
- Allora il secondo (detto **postcondizione**) è vero sui dati di uscita

Queste specifiche semantiche sono dette **assiomatiche**

Nell'esempio del fattoriale:

Precondizione: $n \in \mathbb{N}$

Postcondizione: $fatt(n) = \begin{cases} n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$

Stepwise refinement

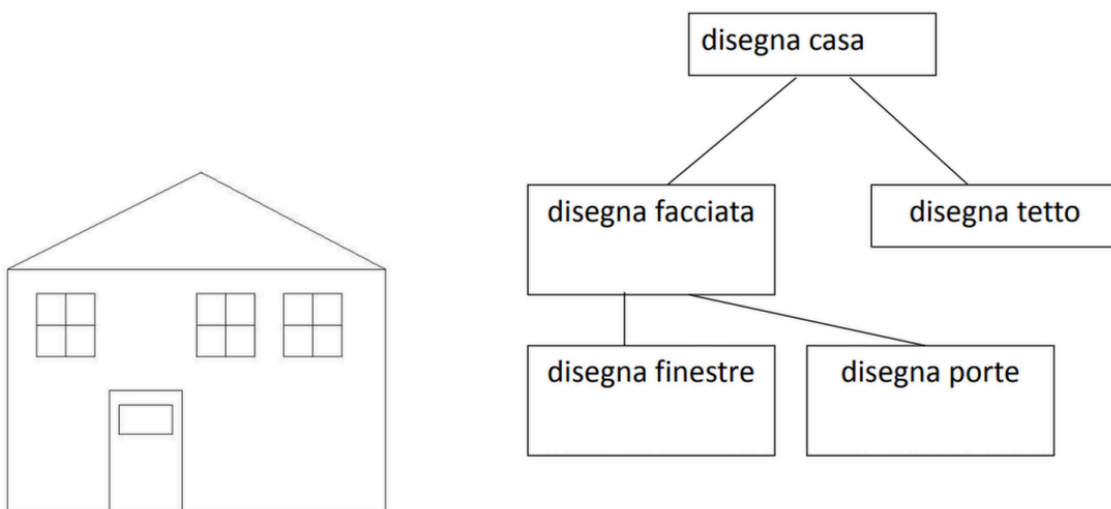
L'astrazione funzionale si è affermata pienamente solo quando emerse una metodologia che mirava a costruire i programmi **progredendo dal generale al particolare**, ossia quella del **stepwise refinement**.

Si caratterizza secondo i seguenti passi:

1. Decomponi il compito P in sotto-compiti $P_1, P_2, \dots, P_n$
2. Ipoteizza di disporre di moduli $M_1, M_2 \dots M_n$ che effettuano le trasformazioni richieste rispettivamente da $P_1, P_2, \dots, P_n$. Tali moduli vengono specificati ma non ancora implementati.
3. Componi un modulo M che assolve al compito P usando i moduli $M_1, M_2 \dots M_n$
4. Applica ricorsivamente la metodologia ai sotto-compiti $P_1, P_2, \dots, P_n$ al fine di definire la realizzazione di $M_1, M_2 \dots M_n$ fino a quando non si ottengono sotto-compiti considerati elementari (o non ulteriormente decomponibili).

≡ Esempio della casa

Tipicamente la dipendenza fra moduli è rappresentata da un albero.



Secondo la metodologia di stepwise refinement, il programmatore è libero di assumere l'esistenza di qualsiasi modulo (detto **stub**, lett. matrice di qualcosa) che si può applicare al particolare sotto-compito e di cui fornisce una specifica, salvo dover poi specificare come quel modulo va realizzato.

Limiti dell'astrazione funzionale

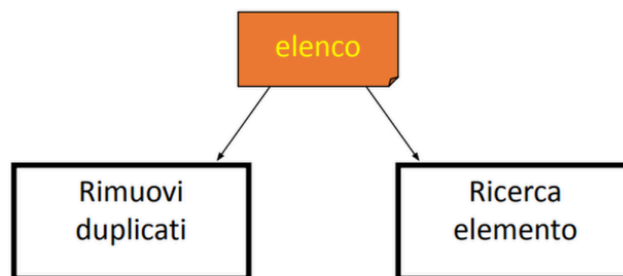
L'astrazione funzionale ha diversi limiti:

- I dettagli relativi alla rappresentazione dei dati di input e output devono essere conosciuti da chi poi andrà a realizzare il modulo (ad esempio un modulo che rimuove i duplicati in un elenco deve sapere se questo è realizzato con un array, un file, etc.)
- La rappresentazione è solitamente condivisa tra diversi moduli, per cui i cambiamenti apportati alla rappresentazione dei dati in input/output a un modulo si possono ripercuotere su molti altri moduli.

≡ Esempio di limite

Limiti dell'astrazione funzionale

Esempio: due moduli operano su uno stesso elenco, uno per rimuovere duplicati e l'altro per ricercare un elemento. Per migliorare l'efficienza dell'ultimo sarebbe opportuno ricorrere a una rappresentazione dell'elenco con tabelle hash. Tuttavia ciò comporta un cambiamento nel primo modulo, che dovrà necessariamente operare la trasformazione (rimozione dei duplicati) in modo differente.



L'astrazione funzionale non permette quindi di sviluppare soluzioni **invarianti ai cambiamenti nei dati** (sono invarianti solo ai cambiamenti nei processi di trasformazione che operano), rendendo quindi difficoltosa la manutenzione delle soluzioni progettate e inappropriata per lo sviluppo di soluzioni a problemi complessi.

Astrazione dati

Per poter risolvere il problema precedente si è inventata l'astrazione dati.

Definizione

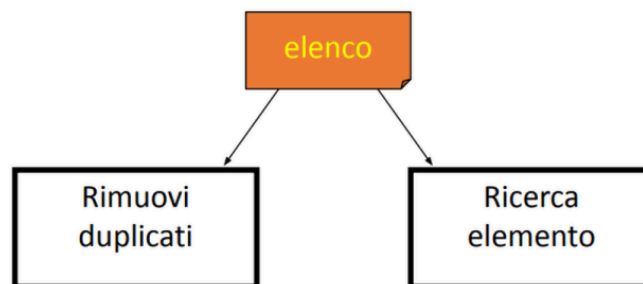
Alla base dell'**astrazione dati** c'è il principio che non si può accedere direttamente alla rappresentazione di un dato, qualunque esso sia, ma solo attraverso un insieme di operazioni considerate lecite.

Questo porta ad un vantaggio non indifferente, ossia un cambiamento nella rappresentazione del dato si ripercuote solo sulle operazioni lecite, che potrebbero subire delle modifiche, mentre non influenzerà il codice che utilizza il dato astratto.

Riprendiamo il caso di prima nei limiti dell'astrazione dati

Astrazione dati

Esempio: Se i moduli 'Rimuovi duplicati' e 'Ricerca elemento' accedono all'elenco attraverso un insieme di operazioni lecite (e.g., 'dammi il prossimo elemento', 'non ci sono più elementi', ecc.) il cambiamento della rappresentazione dell'elenco richiede una riformulazione delle operazioni lecite, ma non influenzerà i due moduli.



Information hiding

Tutte le astrazioni seguono il principio dell'**information hiding**, ossia l'occultamento dei dettagli del processo di trasformazione (**non si dice come farlo**).

Il principio dell'astrazione dati identifica nella rappresentazione del dato l'informazione da nascondere.

Incapsulamento

L'incapsulamento (**encapsulation**) è una tecnica di progettazione che consiste nell'impacchettare (o "racchiudere in capsule") una collezione di entità, creandone una barriera concettuale

Come l'astrazione, l'incapsulamento sotto-intende:

- **Un processo:** l'impacchettamento
- **Una entità:** il "pacchetto" ottenuto

☰ Alcuni esempi di incapsulamento

- Una procedura impacchetta diversi comandi
- Una libreria incapsula diverse funzioni
- Un oggetto incapsula un dato e un insieme di operazioni sul dato

L'incapsulamento non dice come devono essere i contenuti della capsula, che potranno essere

- **Trasparenti:** permettendo di vedere tutto ciò che è stato impacchettato
- **Traslucidi:** permettendo di vedere in modo parziale il contenuto
- **Opachi:** nascondendo tutto il contenuto del pacchetto

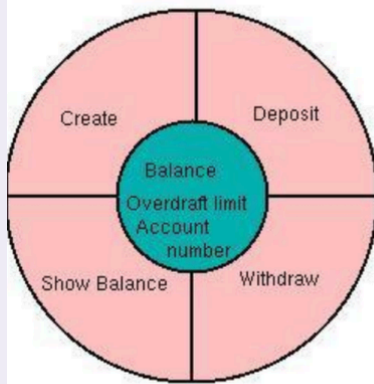
In C++ ad esempio sono le funzioni precedute da `private` o `public`

La combinazione del **principio dell'astrazione dati con la tecnica dell'incapsulamento** suggerisce che:

1. La rappresentazione del dato va nascosta all'interno dell'incapsulamento
2. L'accesso al dato deve passare solo attraverso operazioni lecite
3. Le operazioni lecite, che ovviamente devono avere accesso all'informazione sulla rappresentazione del dato, vanno impacchettate con la rappresentazione del dato stesso

☰ Esempio di unione di astrazione dati e incapsulamento

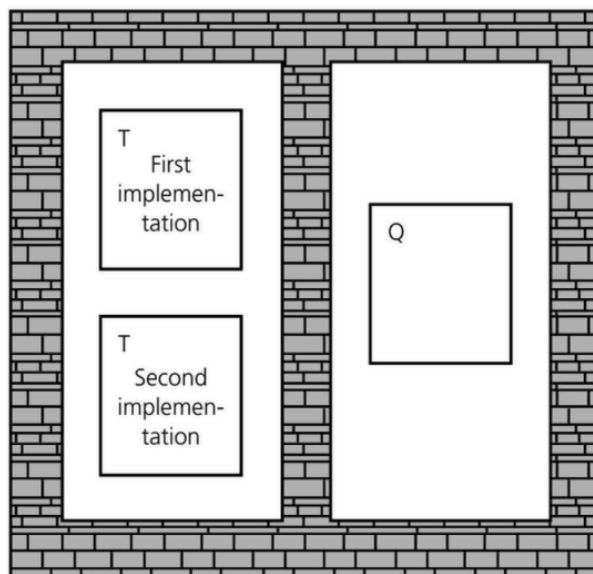
Esempio: il dato “conto corrente” ha una sua rappresentazione interna che permette di memorizzare il saldo (*balance*), il limite fido (*overcraft limit*) e il numero di conto (*account number*). La rappresentazione interna dei tre dati è nascosta. L'accesso alla rappresentazione passa per tre operazioni lecite:



- Creazione conto
- Deposito
- Prelievo
- Stampa saldo

Rappresentazione e operazioni lecite sono impacchettate in un modulo.

T e Q sono isolati: l'implementazione di T non influenza Q



Astrazione dati vs Astrazione funzionale

L'astrazione dati **ricalca ed estende** quella funzionale.

Attualmente la possibilità di effettuare astrazioni di dati è considerata importante almeno quanto quella di definire nuovi operatori con astrazioni funzionali, l'esperienza ha infatti dimostrato che la scelta delle strutture di dati è il primo passo sostanziale per un buon risultato dell'attività di programmazione.

Quindi:

- L'astrazione funzionale stimola gli sforzi per evidenziare operazioni ricorrenti o comunque ben caratterizzate all'interno della soluzione di un dato problema.
- L'astrazione di dati stimola in più gli sforzi per individuare varie organizzazioni di dati più consone alla soluzione del problema.

La progettazione da **function centered** diventa **data centered**.

L'isolamento però di questi due moduli non può essere totale, quindi si va ad utilizzare la **specifica**

I punti di vista dell'astrazione

In generale, le astrazioni supportano la separazione dei diversi interessi di

- **Utenti:** interessati a cosa si astrae (what)
 - **Implementatori:** interessati a come (how) si realizza
- Per questa ragione una definizione di astrazione ha sempre due componenti:
- **Specifica**
 - **Realizzazione**
- Per descrivere una specifica occorre ricorrere a dei linguaggi di specifica, che sono diversi dai linguaggi usati per descrivere le realizzazione delle astrazioni.

Astrazione funzionale

Specifica sintattica e semantica

Definizione di specifica

La specifica, o contratto, descrive come si può interagire con un dato astratto

A sua volta, la specifica si divide in due livelli:

- **Specifica Sintattica:** Stabilisce quali identificatori (nomi) sono associati all'astrazione.
- **Specifica Semantica:** Definisce esattamente il risultato della computazione inclusa nell'astrazione.

Parametrizzazione di un'astrazione

L'efficacia di un'astrazione migliora grazie all'uso di parametri, che facilitano la comunicazione con l'ambiente esterno.

Quando avviene la chiamata di un'astrazione, ogni parametro formale viene associato al rispettivo argomento. Questo avviene tramite due possibili meccanismi:

- **Meccanismi di copia:** Eseguono una copia fisica del valore da passare.
- **Meccanismi definizionali:** Creano un legame diretto tra il parametro formale e la definizione stessa dell'argomento passato.

Astrazione dati

Anche il dato astratto è costituito da una specifica e da una realizzazione.

- La **specifica** descrive il nuovo dato e tutti gli operatori che è possibile applicargli.
- La **realizzazione** definisce come questo nuovo dato e i relativi operatori si appoggiano ai dati e operatori già disponibili a livello inferiore.

Chi programma utilizzando i nuovi dati deve conoscerne perfettamente la specifica, ma può ignorare del tutto le tecniche usate per la realizzazione.

I linguaggi di specifica per i dati astratti si dividono in due filoni principali:

- **Specifiche Assiomatiche:** Usano un linguaggio logico-matematico basato su asserzioni.
- **Specifiche Algebriche:** Usano il linguaggio dell'algebra per definire equazioni tra gli operatori del dato.

Specifiche assiomatiche

Le **specifiche assiomatiche** servono a formalizzare un tipo di dato astratto utilizzando la notazione logico-matematica delle asserzioni.

Una specifica assiomatica si compone rigorosamente di due parti:

1. **Specifica Sintattica (Signature):** Questa parte fornisce l'interfaccia formale del dato astratto, definendo:
 - L'elenco dei nomi dei domini e delle operazioni specifiche previste per quel tipo di dato.
 - I domini di partenza (input) e i domini di arrivo (output) per ogni singolo nome di operatore.
2. **Specifica Semantica:** Questa parte definisce il comportamento e il significato delle operazioni:
 - Associa un insieme matematico preciso a ogni nome di tipo che è stato introdotto nella specifica sintattica.
 - Associa una funzione logica a ogni nome di operatore, esplicitando due vincoli (o condizioni) fondamentali:
 - **Precondizione:** Definisce quando l'operatore è effettivamente applicabile (i requisiti sui dati di partenza).
 - **Postcondizione:** Stabilisce la relazione esatta che lega gli argomenti di input al risultato finale prodotto

Limitazioni delle specifiche assiomatiche

Il metodo di specifica assiomatica è rigorosamente preciso nella sua definizione sintattica, ma si rivela spesso informale nella sua parte semantica, ricorrendo a volte al linguaggio

naturale per descrivere i comportamenti.

A causa di questa informalità, non consente di caratterizzare un tipo astratto in maniera totalmente precisa.

I difetti principali riguardano l'impossibilità di definire esattamente l'insieme dei valori generabili tramite l'applicazione degli operatori e l'incapacità di stabilire quando due diverse sequenze di operatori producono il medesimo valore finale.

Per superare queste limitazioni, si ricorre alle specifiche algebriche.

Specifiche algebriche

Le specifiche algebriche poggiano sui fondamenti dell'algebra, allontanandosi dalla logica.

Questo approccio definisce un dato astratto come un'**algebra eterogenea**, ovvero una collezione composta da insiemi diversi sui quali agiscono diverse operazioni.

Questo concetto si contrappone all'algebra tradizionale, detta omogenea, che si basa su un unico insieme abbinato a diverse operazioni (come l'insieme dei numeri interi $\mathbb{Z}$ con le operazioni di addizione e moltiplicazione).

Una specifica algebrica è divisa rigidamente in tre componenti:

1. **Specifica sintattica:** Elenca esplicitamente i nomi del tipo, tutte le sue operazioni e il tipo associato agli argomenti di tali operazioni. Nel caso in cui un'operazione funga da funzione, ne viene indicato anche il codominio (range).
2. **Specifica semantica:** È costituita da un insieme di equazioni algebriche. Queste equazioni descrivono in modo rigoroso le proprietà delle operazioni, mantenendosi totalmente indipendenti da quella che sarà la reale rappresentazione dei dati in memoria.
3. **Specifica di restrizione:** Definisce le varie condizioni che devono essere obbligatoriamente soddisfatte prima dell'applicazione delle operazioni oppure al loro termine. (Alcune metodologie scelgono di accorpare queste restrizioni direttamente all'interno delle specifiche semantiche).

Un aspetto molto potente delle specifiche algebriche è la semplicità del loro linguaggio, specialmente se confrontato con i normali linguaggi di programmazione procedurale.

L'intero linguaggio di specifica poggia su sole cinque primitive basilari:

1. La composizione funzionale (indicata con il simbolo $\rightarrow$).
2. La relazione di eguaglianza.
3. La costante booleana `true`.
4. La costante booleana `false`.
5. La disponibilità di un numero illimitato di variabili libere.

Inoltre, per comodità, si assumono già come predefiniti i valori booleani, i valori interi e la funzione condizionale `if then else`.

Quest'ultima funzione è descrivibile dalle equazioni matematiche `if then else (true, q, r) = q` e `if then else (false, q, r) = r`. Essendo fondamentale, viene

comunemente scritta e utilizzata come un operatore infisso nella forma `if p then q else r.`

📌 I tre pilastri della specifica algebrica

I **tre** pilastri della specifica algebrica sono:

SORT	AZIONI (specifica sintattica)	EQUAZIONI (specifica semantica)
Quali tipi di dati stiamo usando? (es. stack, item, boolean)	La firma delle funzioni. Nome funzione, argomenti di input e di output.	Le regole del gioco, qui scriviamo le verità attraverso l'utilizzo di equazioni.

Specifica algebrica di una Pila

☰ Specifica algebrica di una Pila

Specifica algebrica di *Pila*

Specifica sintattica

sorts: stack, item, boolean

operations:

newstack() -> stack

push(stack, item) -> stack

pop(stack) -> stack

top(stack) -> item

isnew(stack) -> boolean

Ogni insieme è detto un **sort** (letteralmente “tipo”) dell'algebra eterogenea.

I sort **item** e **boolean** sono **ausiliari** alla definizione di stack.

Specifica algebrica di *Pila*

Specifica semantica

declare **stk**: stack, **i**: item

pop(push(stk, i)) = stk

top(push(stk, i)) = i

isnew(newstack) = true

isnew(push(stk, i)) = false

Nella specifica semantica occorre **dichiarare** i parametri su cui lavorano gli operatori.

Specifica algebrica di *Pila*

Specifica di restrizione

restrictions

pop(newstack) = error

top(newstack) = error

L'error nelle specifiche algebriche va ad inserire equazioni che non possono essere utilizzate (gestiscono quindi delle restrizioni)

Indicazioni importanti

- Nelle specifiche semantiche è importante indicare l'insieme minimale di equazioni (dette **assiomi**) a partire dalle quali possiamo derivare tutte le altre.
- Le specifiche semantiche si diranno **incomplete** se non permetteranno di derivare tutte le verità (equazioni) desiderate dell'algebra specificata.
- Le specifiche semantiche si diranno **inconsistenti** (o contraddittorie) se permetteranno di derivare delle equazioni indesiderate, cioè considerate false.
- Le specifiche semantiche si diranno **ridondanti** se alcune delle equazioni sono ricavabili dalle altre

Costruttori e osservazioni

Scrivere delle specifiche semantiche complete, consistenti e non ridondanti può non essere un compito semplice. Per questo conviene introdurre una metodologia, che si basa sulla distinzione degli operatori di un dato astratto in:

- **Costruttori**, che creano o istanziano il dato astratto
- **Osservazioni**, che ritrovano informazioni sul dato astratto

Il comportamento di una astrazione dati può essere specificata riportando il valore di ciascuna osservazione applicata a ciascun costruttore. Questa informazione è organizzata in modo naturale in una matrice, con i costruttori lungo una dimensione e le osservazioni lungo l'altra.

☰ Costruttori della Pila

<i>osservazioni</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>pop(stk')</i>	error	stk
<i>top(stk')</i>	error	i
<i>isnew(stk')</i>	true	false

Osservazioni binarie

Tutte le osservazioni viste finora sono **unarie**, nel senso che esse osservano un singolo valore del dato astratto. Spesso è necessario disporre di osservazioni più complesse. Per confrontare due valori è necessario osservare due istanze del dato astratto. Questo complica la specifica, perché il valore dell'osservazione dev'essere definito per tutte le **combinazioni di costruttori** possibili per i valori astratti che si devono confrontare

☰ Osservazione binaria sulla Pila

Esempio: predicato *equal(l, m)* che è vero solo se le due pile contengono gli stessi elementi nello stesso ordine.

<i>Costruttore di m</i>	<i>Costruttore di l</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>newstack</i>	true	false
<i>push(stk', i')</i>	false	$i=i'$ and $\text{equal}(\text{stk}, \text{stk}')$

Questa tabella può essere vista come l'aggiunta di una terza dimensione alla tabella di base per le osservazioni unarie.

Osservazione binaria: *equal(stk', m)*

Questa terza dimensione può essere rimossa andando a classificare esplicitamente solo il primo argomento dell'osservazione e referenziando in modo astratto il secondo argomento mediante una *variabile libera*.

<i>osservazione</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>equal(stk', m)</i>	<i>isnew(m)</i>	not <i>isnew(m)</i> and $i=\text{top}(m)$ and $\text{equal}(\text{stk}, \text{pop}(m))$

La scelta dei costruttori

Talvolta, l'applicazione della metodologia per la sintesi di specifiche algebriche può comportare delle difficoltà riguardo al discernimento di cosa è costruttore e cosa operazione.

Nello scegliere i costruttori adotteremo il **criterio di minimalità**, cioè l'insieme dei costruttori dev'essere il più piccolo insieme di operatori necessario a costruire tutti i possibili valori per un certo dato astratto

Astrazione di controllo

L'astrazione di controllo si riferisce alla possibilità di specificare un modulo software che esegue delle operazioni in un ordine, nascondendo i dettagli su come il mantenimento dell'ordine è ottenuto

In sostanza:

- Il modulo software deve essere parametrizzato rispetto alle operazioni da eseguire
- Il modulo software è associato a un controllo di sequenza (**control flow**)
- I dettagli di come il controllo di sequenza è garantito non sono visibili al consumatore (fruitore) del modulo

☰ Modulo while()

Un modulo che ha in input un'espressione e un comando, iterando l'esecuzione del comando fintanto che l'espressione è vera è un modulo che implementa il costrutto di controllo **while()**.

Sappiamo benissimo che l'implementazione del while è un'implementazione **nascosta**.

Può essere realizzato iterativamente in vari modi:

- | | |
|-----------------------------|-----------------------------|
| 1. Valuta espressione | 1. Valuta espressione |
| 2. Se true allora vai a 4 | 2. Se false allora vai a 5 |
| 3. Esci dal modulo (return) | 3. Esegui istruzione |
| 4. Esegui istruzione | 4. Vai a 1 |
| 5. Vai a 1 | 5. Esci dal modulo (return) |

Esempio (cont.): o persino ricorsivamente!

1. Valuta espressione
2. Se true allora
 - a) Esegui istruzione
 - b) while(espressione, istruzione).
3. Esci dal modulo (return)

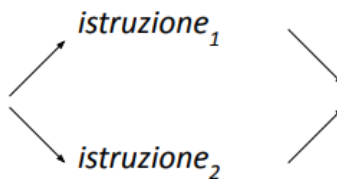
Tutte queste realizzazioni (iterative o ricorsive) sono corrette rispetto alla semantica dell'astrazione di controllo while. All'utente del modulo non importa sapere quale delle realizzazioni è stata adottata.

Nella programmazione sequenziale l'ordine di esecuzione delle istruzioni è **totale**, mentre nella programmazione concorrente/parallela l'ordine di esecuzione può essere **parziale**

☰ L'astrazione di controllo

fork-join(istruzione1, istruzione2)

che prevede l'esecuzione parallela di *istruzione1* e *istruzione2* definisce una relazione d'ordine parziale fra le due istruzioni.



I linguaggi di specifica utilizzati per l'astrazione di controllo si basano sulla definizione di **relazioni di precedenza** fra istruzioni che permettono di stabilire sia ordinamenti totali che ordinamenti parziali.

La combinazione dell'astrazione dati con l'astrazione di controllo permette di sviluppare soluzioni software **invarianti** non solo al cambiamento della realizzazione dei dati ma anche al **cambiamento dei meccanismi di visita del dato astratto**.

Ciò risulta molto utile poiché l'iterazione su collezioni di dati è un compito comune e ripetitivo e l'evidenza empirica indica che spesso si commettono errori nella inizializzazione dell'iterazione e nella scrittura delle condizioni di stop

☰ Esempio completo di astrazione di controllo

Esempio: Un dato astratto che **contiene** un insieme di valori può mettere a disposizione due operatori, **next** che restituisce un valore e il predicato **hasNext** che è vero se ci sono ancora altri valori da selezionare. L'astrazione di controllo *for-each(dato astratto, istruzione)* utilizzerà questi due operatori per eseguire l'*istruzione* su ogni valore contenuto nel dato astratto. **L'informazione sull'ordine con cui i valori sono considerati (cioè sul protocollo di iterazione) è nascosta all'utente del *for-each*.**

Andando ancora oltre, si può ipotizzare che il dato astratto deleghi un modulo **iteratore** a fornire i servizi richiesti dal *for-each*. In questo modo i meccanismi di visita sono tutti incapsulati in un modulo.

Ancora una volta, la tecnica dell'incapsulamento viene in aiuto al progettista che vuole applicare una qualche forma di astrazione.

ATTENZIONE

Alcuni esempi sono stati omessi poichè potrebbero risultare troppo prolissi, nel caso li si vogliano visionare si possono trovare nel file seguente, estratto dalle slide del prof:

[1.1 - \(Slide Esempi\) Paradigmi di programmazione e astrazione.pdf](#)